

1

INTRODUCTION

*Begin at the beginning . . . and go on till you come to the end:
then stop.*

—Lewis Carroll

Chapter Outline

- 1.1 Introduction
 - 1.2 Network Flow Problems
 - 1.3 Applications
 - 1.4 Summary
-

1.1 INTRODUCTION

Everywhere we look in our daily lives, networks are apparent. Electrical and power networks bring lighting and entertainment into our homes. Telephone networks permit us to communicate with each other almost effortlessly within our local communities and across regional and international borders. National highway systems, rail networks, and airline service networks provide us with the means to cross great geographical distances to accomplish our work, to see our loved ones, and to visit new places and enjoy new experiences. Manufacturing and distribution networks give us access to life's essential foodstock and to consumer products. And computer networks, such as airline reservation systems, have changed the way we share information and conduct our business and personal lives.

In all of these problem domains, and in many more, we wish to move some entity (electricity, a consumer product, a person or a vehicle, a message) from one point to another in an underlying network, and to do so as efficiently as possible, both to provide good service to the users of the network and to use the underlying (and typically expensive) transmission facilities effectively. In the most general sense, this objective is what this book is all about. We want to learn how to model application settings as mathematical objects known as network flow problems and to study various ways (algorithms) to solve the resulting models.

Network flows is a problem domain that lies at the cusp between several fields of inquiry, including applied mathematics, computer science, engineering, management, and operations research. The field has a rich and long tradition, tracing its roots back to the work of Gustav Kirchhof and other early pioneers of electrical engineering and mechanics who first systematically analyzed electrical circuits. This early work set the foundations of many of the key ideas of network flow theory and established networks (graphs) as useful mathematical objects for representing many

physical systems. Much of this early work was descriptive in nature, answering such questions as: If we apply a set of voltages to a given network, what will be the resulting current flow? The set of questions that we address in this book are a bit different: If we have alternative ways to use a network (i.e., send flow), which alternative will be most cost-effective? Our intellectual heritage for answering such questions is much more recent and can be traced to the late 1940s and early 1950s when the research and practitioner communities simultaneously developed optimization as an independent field of inquiry and launched the computer revolution, leading to the powerful instruments we know today for performing scientific and managerial computations.

For the most part, in this book we wish to address the following basic questions:

1. *Shortest path problem.* What is the best way to traverse a network to get from one point to another as cheaply as possible?
2. *Maximum flow problem.* If a network has capacities on arc flows, how can we send as much flow as possible between two points in the network while honoring the arc flow capacities?
3. *Minimum cost flow problem.* If we incur a cost per unit flow on a network with arc capacities and we need to send units of a good that reside at one or more points in the network to one or more other points, how can we send the material at minimum possible cost?

In the sense of traditional applied and pure mathematics, each of these problems is trivial to solve. It is not very difficult (but not at all obvious for the later two problems) to see that we need only consider a finite number of alternatives for each problem. So a traditional mathematician might say that the problems are well solved: Simply enumerate the set of possible solutions and choose the one that is best. Unfortunately, this approach is far from pragmatic, since the number of possible alternatives can be very large—more than the number of atoms in the universe for many practical problems! So instead, we would like to devise algorithms that are in a sense “good,” that is, whose computation time is small, or at least reasonable, for problems met in practice. One way to ensure this objective is to devise algorithms whose running time is guaranteed not to grow very fast as the underlying network becomes larger (the computer science, operations research, and applied mathematics communities refer to the development of algorithms with such performance guarantees as *worst-case analysis*). Developing algorithms that are good in this sense is another major theme throughout this book, and our development builds heavily on the theory of computational complexity that began to develop within computer science, applied mathematics, and operations research circles in the 1970s, and has flourished ever since.

The field of computational complexity theory combines both craftsmanship and theory; it builds on a confluence of mathematical insight, creative algorithm design, and the careful, and often very clever use of data structures to devise solution methods that are provably good in the sense that we have just mentioned. In the field of network flows, researchers devised the first, seminal contributions of this nature in the 1950s before the field of computational complexity theory even existed as a separate discipline as we know it today. And throughout the last three decades,

researchers have made a steady stream of innovations that have resulted in new solution methods and in improvements to known methods. In the past few years, however, researchers have made contributions to the design and analysis of network flow algorithms with improved worst-case performance guarantees at an explosive, almost dizzying pace; moreover, these contributions were very surprising: Throughout the 1950s, 1960s, and 1970s, network flows had evolved into a rather mature field, so much so that most of the research and practitioner communities believed that the core models that we study in this book were so very well understood that further innovations would be hard to come by and would be few and far between. As it turns out, nothing could have been further from the truth.

Our presentation is intended to reflect these new developments; accordingly, we place a heavy emphasis on designing and analyzing good algorithms for solving the core optimization models that arise in the context of network flows. Our intention is to bring together and synthesize the many new contributions concerning efficient network flow algorithms with traditional material that has evolved over the past four decades. We have attempted to distill and highlight some of the essential core ideas (e.g., scaling and potential function arguments) that underlie many of the recent innovations and in doing so to give a unified account of the many algorithms that are now available. We hope that this treatment will provide our readers not only with an accessible entrée to these exciting new developments, but also with an understanding of the most recent and advanced contributions from the literature. Although we are bringing together ideas and methodologies from applied mathematics, computer science, and operations research, our approach has a decidedly computer science orientation as applied to certain types of models that have traditionally arisen in the context of managing a variety of operational systems (the foodstuff of operations research).

We feel that a full understanding of network flow algorithms and a full appreciation for their use requires more than an in-depth knowledge of good algorithms for core models. Consequently, even though this topic is our central thrust, we also devote considerable attention to describing applications of network flow problems. Indeed, we feel that our discussion of applications throughout the text, in the exercises, and in a concluding chapter is one of the major distinguishing features of our coverage.

We have not adopted a linear programming perspective throughout the book, however, because we feel there is much to be gained from a more direct approach, and because we would like the material we cover to be readily accessible to readers who are not optimization specialists. Moreover, we feel that an understanding of network flow problems from first principles provides a useful concrete setting from which to draw considerable insight about more general linear programs.

Similarly, since several important variations of the basic network flow problems are important in practice, or in placing network flows in the broader context of the field of combinatorial optimization, we have also included several chapters on additional topics: assignments and matchings, minimum spanning trees, models with convex (instead of linear) costs, networks with losses and gains, and multicommodity flows. In each of these chapters we have not attempted to be comprehensive, but rather, have tried to provide an introduction to the essential ideas of the topics.

The Lagrangian relaxation chapter permits us to show how the core network

models arise in broader problem contexts and how the algorithms that we have developed for the core models can be used in conjunction with other methods to solve more complex problems that arise frequently in practice. In particular, this discussion permits us to introduce and describe the basic ideas of decomposition methods for several important network optimization models—constrained shortest paths, the traveling salesman problem, vehicle routing problem, multicommodity flows, and network design.

Since the proof of the pudding is in the eating, we have also included a chapter on some aspects of computational testing of algorithms. We devote much of our discussion to devising the best possible algorithms for solving network flow problems, in the theoretical sense of computational complexity theory. Although the theoretical model of computation that we are using has proven to be a valuable guide for modeling and predicting the performance of algorithms in practice, it is not a perfect model, and therefore algorithms that are not theoretically superior often perform best in practice. Although empirical testing of algorithms has traditionally been a valuable means for investigating algorithmic ideas, the applied mathematics, computer science, and operations research communities have not yet reached a consensus on how to measure algorithmic performance empirically. So in this chapter we not only report on computational experience with an algorithm we have presented, but also offer some thoughts on how to measure computational performance and compare algorithms.

1.2 NETWORK FLOW PROBLEMS

In this section we introduce the network flow models we study in this book, and in the next section we present several applications that illustrate the practical importance of these models. In both the text and exercises throughout the remaining chapters, we introduce many other applications. In particular, Chapter 19 contains a more comprehensive summary of applications with illustrations drawn from several specialties in applied mathematics, engineering, logistics, manufacturing, and the physical sciences.

Minimum Cost Flow Problem

The minimum cost flow model is the most fundamental of all network flow problems. Indeed, we devote most of this book to the minimum cost flow problem, special cases of it, and several of its generalizations. The problem is easy to state: We wish to determine a least cost shipment of a commodity through a network in order to satisfy demands at certain nodes from available supplies at other nodes. This model has a number of familiar applications: the distribution of a product from manufacturing plants to warehouses, or from warehouses to retailers; the flow of raw material and intermediate goods through the various machining stations in a production line; the routing of automobiles through an urban street network; and the routing of calls through the telephone system. As we will see later in this chapter and in Chapters 9 and 19, the minimum cost flow model also has many less transparent applications.

In this section we present a mathematical programming formulation of the minimum cost flow problem and then describe several of its specializations and

variants as well as other basic models that we consider in later chapters. We assume our readers are familiar with the basic notation and definitions of graph theory; those readers without this background might consult Section 2.2 for a brief account of this material.

Let $G = (N, A)$ be a directed network defined by a set N of n nodes and a set A of m directed arcs. Each arc $(i, j) \in A$ has an associated cost c_{ij} that denotes the cost per unit flow on that arc. We assume that the flow cost varies linearly with the amount of flow. We also associate with each arc $(i, j) \in A$ a capacity u_{ij} that denotes the maximum amount that can flow on the arc and a lower bound l_{ij} that denotes the minimum amount that must flow on the arc. We associate with each node $i \in N$ an integer number $b(i)$ representing its supply/demand. If $b(i) > 0$, node i is a *supply node*; if $b(i) < 0$, node i is a *demand node* with a demand of $-b(i)$; and if $b(i) = 0$, node i is a *transshipment node*. The decision variables in the minimum cost flow problem are arc flows and we represent the flow on an arc $(i, j) \in A$ by x_{ij} . The minimum cost flow problem is an optimization model formulated as follows:

$$\text{Minimize } \sum_{(i,j) \in A} c_{ij}x_{ij} \quad (1.1a)$$

subject to

$$\sum_{\{j:(i,j) \in A\}} x_{ij} - \sum_{\{j:(j,i) \in A\}} x_{ji} = b(i) \quad \text{for all } i \in N, \quad (1.1b)$$

$$l_{ij} \leq x_{ij} \leq u_{ij} \quad \text{for all } (i,j) \in A, \quad (1.1c)$$

where $\sum_{i=1}^n b(i) = 0$. In matrix form, we represent the minimum cost flow problem as follows:

$$\text{Minimize } cx \quad (1.2a)$$

subject to

$$Nx = b, \quad (1.2b)$$

$$l \leq x \leq u. \quad (1.2c)$$

In this formulation, N is an $n \times m$ matrix, called the *node-arc incidence matrix* of the minimum cost flow problem. Each column N_{ij} in the matrix corresponds to the variable x_{ij} . The column N_{ij} has a $+1$ in the i th row, a -1 in the j th row; the rest of its entries are zero.

We refer to the constraints in (1.1b) as *mass balance constraints*. The first term in this constraint for a node represents the total *outflow* of the node (i.e., the flow emanating from the node) and the second term represents the total *inflow* of the node (i.e., the flow entering the node). The mass balance constraint states that the outflow minus inflow must equal the supply/demand of the node. If the node is a supply node, its outflow exceeds its inflow; if the node is a demand node, its inflow exceeds its outflow; and if the node is a transshipment node, its outflow equals its inflow. The flow must also satisfy the lower bound and capacity constraints (1.1c), which we refer to as *flow bound constraints*. The flow bounds typically model physical capacities or restrictions imposed on the flows' operating ranges. In most applications, the lower bounds on arc flows are zero; therefore, if we do not state lower bounds for any problem, we assume that they have value zero.

In most parts of the book we assume that the data are integral (i.e., all arc capacities, arc costs, and supplies/demands of nodes are integral). We refer to this assumption as the *integrality assumption*. The integrality assumption is not restrictive for most applications because we can always transform rational data to integer data by multiplying them by a suitably large number. Moreover, we necessarily need to convert irrational numbers to rational numbers to represent them on a computer.

The following special versions of the minimum cost flow problem play a central role in the theory and applications of network flows.

Shortest path problem. The shortest path problem is perhaps the simplest of all network flow problems. For this problem we wish to find a path of minimum cost (or length) from a specified *source node* s to another specified *sink node* t , assuming that each arc $(i, j) \in A$ has an associated cost (or length) c_{ij} . Some of the simplest applications of the shortest path problem are to determine a path between two specified nodes of a network that has minimum length, or a path that takes least time to traverse, or a path that has the maximum reliability. As we will see in our later discussions, this basic model has applications in many different problem domains, such as equipment replacement, project scheduling, cash flow management, message routing in communication systems, and traffic flow through congested cities. If we set $b(s) = 1$, $b(t) = -1$, and $b(i) = 0$ for all other nodes in the minimum cost flow problem, the solution to the problem will send 1 unit of flow from node s to node t along the shortest path. The shortest path problem also models situations in which we wish to send flow from a single-source node to a single-sink node in an uncapacitated network. That is, if we wish to send v units of flow from node s to node t and the capacity of each arc of the network is at least v , we would send the flow along a shortest path from node s to node t . If we want to determine shortest paths from the source node s to every other node in the network, then in the minimum cost flow problem we set $b(s) = (n - 1)$ and $b(i) = -1$ for all other nodes. [We can set each arc capacity u_{ij} to any number larger than $(n - 1)$.] The minimum cost flow solution would then send unit flow from node s to every other node i along a shortest path.

Maximum flow problem. The maximum flow problem is in a sense a complementary model to the shortest path problem. The shortest path problem models situations in which flow incurs a cost but is not restricted by any capacities; in contrast, in the maximum flow problem flow incurs no costs but is restricted by flow bounds. The maximum flow problem seeks a feasible solution that sends the maximum amount of flow from a specified source node s to another specified sink node t . If we interpret u_{ij} as the maximum flow rate of arc (i, j) , the maximum flow problem identifies the maximum steady-state flow that the network can send from node s to node t per unit time. Examples of the maximum flow problem include determining the maximum steady-state flow of (1) petroleum products in a pipeline network, (2) cars in a road network, (3) messages in a telecommunication network, and (4) electricity in an electrical network. We can formulate this problem as a minimum cost flow problem in the following manner. We set $b(i) = 0$ for all $i \in N$, $c_{ij} = 0$ for all $(i, j) \in A$, and introduce an additional arc (t, s) with cost $c_{ts} = -1$ and flow bound $u_{ts} = \infty$. Then the minimum cost flow solution maximizes the flow on arc (t, s) ; but

since any flow on arc (t, s) must travel from node s to node t through the arcs in A [since each $b(i) = 0$], the solution to the minimum cost flow problem will maximize the flow from node s to node t in the original network.

Assignment problem. The data of the assignment problem consist of two equally sized sets N_1 and N_2 (i.e., $|N_1| = |N_2|$), a collection of pairs $A \subseteq N_1 \times N_2$ representing possible assignments, and a cost c_{ij} associated with each element $(i, j) \in A$. In the assignment problem we wish to pair, at minimum possible cost, each object in N_1 with exactly one object in N_2 . Examples of the assignment problem include assigning people to projects, jobs to machines, tenants to apartments, swimmers to events in a swimming meet, and medical school graduates to available internships. The assignment problem is a minimum cost flow problem in a network $G = (N_1 \cup N_2, A)$ with $b(i) = 1$ for all $i \in N_1$, $b(i) = -1$ for all $i \in N_2$, and $u_{ij} = 1$ for all $(i, j) \in A$.

Transportation problem. The transportation problem is a special case of the minimum cost flow problem with the property that the node set N is partitioned into two subsets N_1 and N_2 (of possibly unequal cardinality) so that (1) each node in N_1 is a supply node, (2) each node N_2 is a demand node, and (3) for each arc (i, j) in A , $i \in N_1$ and $j \in N_2$. The classical example of this problem is the distribution of goods from warehouses to customers. In this context the nodes in N_1 represent the warehouses, the nodes in N_2 represent customers (or, more typically, customer zones), and an arc (i, j) in A represents a distribution channel from warehouse i to customer j .

Circulation problem. The circulation problem is a minimum cost flow problem with only transshipment nodes; that is, $b(i) = 0$ for all $i \in N$. In this instance we wish to find a feasible flow that honors the lower and upper bounds l_{ij} and u_{ij} imposed on the arc flows x_{ij} . Since we never introduce any exogenous flow into the network or extract any flow from it, all the flow circulates around the network. We wish to find the circulation that has the minimum cost. The design of a routing schedule of a commercial airline provides one example of a circulation problem. In this setting, any airplane circulates among the airports of various cities; the lower bound l_{ij} imposed on an arc (i, j) is 1 if the airline needs to provide service between cities i and j , and so must dispatch an airplane on this arc (actually, the nodes will represent a combination of both a physical location and a time of day so that an arc connects, for example, New York City at 8 A.M. with Boston at 9 A.M.).

In this book, we also study the following generalizations of the minimum cost flow problem.

Convex cost flow problems. In the minimum cost flow problem, we assume that the cost of the flow on any arc varies linearly with the amount of flow. Convex cost flow problems have a more general cost structure: The cost is a convex function of the amount of flow. Flow costs vary in a convex manner in numerous problem settings, including (1) power losses in an electrical network due to resistance, (2) congestion costs in a city transportation network, and (3) expansion costs of a communication network.

Generalized flow problems. In the minimum cost flow problem, arcs conserve flows (i.e., the flow entering an arc equals the flow leaving the arc). In generalized flow problems, arcs might “consume” or “generate” flow. If x_{ij} units of flow enter an arc (i, j) , then $\mu_{ij}x_{ij}$ units arrive at node j ; μ_{ij} is a positive *multiplier* associated with the arc. If $0 < \mu_{ij} < 1$, the arc is *lossy*, and if $1 < \mu_{ij} < \infty$, the arc is *gainy*. Generalized network flow problems arise in several application contexts: for example, (1) power transmission through electric lines, with power lost with distance traveled, (2) flow of water through pipelines or canals that lose water due to seepage or evaporation, (3) transportation of a perishable commodity, and (4) cash management scenarios in which arcs represent investment opportunities and multipliers represent appreciation or depreciation of an investment’s value.

Multicommodity flow problems. The minimum cost flow problem models the flow of a single commodity over a network. Multicommodity flow problems arise when several commodities use the same underlying network. The commodities may either be differentiated by their physical characteristics or simply by their origin–destination pairs. Different commodities have different origins and destinations, and commodities have separate mass balance constraints at each node. However, the sharing of the common arc capacities binds the different commodities together. In fact, the essential issue addressed by the multicommodity flow problem is the allocation of the capacity of each arc to the individual commodities in a way that minimizes overall flow costs. Multicommodity flow problems arise in many practical situations, including (1) the transportation of passengers from different origins to different destinations within a city; (2) the routing of nonhomogeneous tankers (nonhomogeneous in terms of speed, carrying capability, and operating costs); (3) the worldwide shipment of different varieties of grains (such as corn, wheat, rice, and soybeans) from countries that produce grains to those that consume it; and (4) the transmission of messages in a communication network between different origin–destination pairs.

Other Models

In this book we also study two other important network models: the *minimum spanning tree problem* and the *matching problem*. Although these two models are not flow problems per se, because of their practical and mathematical significance and because of their close connection with several flow problems, we have included them as part of our treatment of network flows.

Minimum spanning tree problem. A spanning tree is a tree (i.e., a connected acyclic graph) that spans (touches) all the nodes of an undirected network. The cost of a spanning tree is the sum of the costs (or lengths) of its arcs. In the minimum spanning tree problem, we wish to identify a spanning tree of minimum cost (or length). The applications of the minimum spanning tree problem are varied and include (1) constructing highways or railroads spanning several cities; (2) laying pipelines connecting offshore drilling sites, refineries, and consumer markets; (3) designing local access networks; and (4) making electric wire connections on a control panel.

Matching problems. A *matching* in a graph $G = (N, A)$ is a set of arcs with the property that every node is incident to at most one arc in the set; thus a matching induces a pairing of (some of) the nodes in the graph using the arcs in A . In a matching, each node is matched with at most one other node, and some nodes might not be matched with any other node. The *matching problem* seeks a matching that optimizes some criteria. Matching problems on a bipartite graphs (i.e., those with two sets of nodes and with arcs that join only nodes between the two sets, as in the assignment and transportation problems) are called *bipartite matching problems*, and those on nonbipartite graphs are called *nonbipartite matching problems*. There are two additional ways of categorizing matching problems: *cardinality matching problems*, which maximize the number of pairs of nodes matched, and *weighted matching problems*, which maximize or minimize the weight of the matching. The weighted matching problem on a bipartite graph is also known as the *assignment problem*. Applications of matching problems arise in matching roommates to hostels, matching pilots to compatible airplanes, scheduling airline crews for available flight legs, and assigning duties to bus drivers.

1.3 APPLICATIONS

Networks are pervasive. They arise in numerous application settings and in many forms. Physical networks are perhaps the most common and the most readily identifiable classes of networks; and among physical networks, transportation networks are perhaps the most visible in our everyday lives. Often, these networks model homogeneous facilities such as railbeds or highways. But on other occasions, they correspond to composite entities that model, for example, complex distribution and logistics decisions. The traditional operations research “transportation problem” is illustrative. In the transportation problem, a shipper with inventory of goods at its warehouses must ship these goods to geographically dispersed retail centers, each with a given customer demand, and the shipper would like to meet these demands incurring the minimum possible transportation costs. In this setting, a transportation link in the underlying network might correspond to a complex distribution channel with, for example, a trucking shipment from the warehouse to a railhead, a rail shipment, and another trucking leg from the destination rail yard to the customer’s site.

Physical networks are not limited to transportation settings; they also arise in several other disciplines of applied science and engineering, such as mathematics, chemistry, and electrical, communications, mechanical, and civil engineering. When physical networks occur in these different disciplines, their nodes, arcs, and flows model many different types of physical entities. For example, in a typical communication network, nodes will represent telephone exchanges and transmission facilities, arcs will denote copper cables or fiber optic links, and flow would signify the transmission of voice messages or of data. Figure 1.1 shows some typical associations for the nodes, arcs, and flows in a variety of physical networks.

Network flow problems also arise in surprising ways for problems that on the surface might not appear to involve networks at all. Sometimes these applications are linked to a physical entity, and at other times they are not. Sometimes the nodes and arcs have a temporal dimension that models activities that take place over time.

Applications	Physical analog of nodes	Physical analog of arcs	Flow
Communication systems	Telephone exchanges, computers, transmission facilities, satellites	Cables, fiber optic links, microwave relay links	Voice messages, data, video transmissions
Hydraulic systems	Pumping stations, reservoirs, lakes	Pipelines	Water, gas, oil, hydraulic fluids
Integrated computer circuits	Gates, registers, processors	Wires	Electrical current
Mechanical systems	Joints	Rods, beams, springs	Heat, energy
Transportation systems	Intersections, airports, rail yards	Highways, railbeds, airline routes	Passengers, freight, vehicles, operators

Figure 1.1 Ingredients of some common physical networks.

Many scheduling applications have this flavor. In any event, networks model a variety of problems in project, machine, and crew scheduling; location and layout theory; warehousing and distribution; production planning and control; and social, medical, and defense contexts. Indeed, these various applications of network flow problems seem to be more widespread than are the applications of physical networks. We present many such applications throughout the text and in the exercises; Chapter 19, in particular, brings together and summarizes many applications. In the following discussion, to set a backdrop for the next few chapters, we describe several sample applications that are intended to illustrate a range of problem contexts and to be suggestive of how network flow problems arise in practice. This set of applications provides at least one example of each of the network models that we introduced in the preceding section.

Application 1.1 Reallocation of Housing

A housing authority has a number of houses at its disposal that it lets to tenants. Each house has its own particular attributes. For example, a house might or might not have a garage, it has a certain number of bedrooms, and its rent falls within a particular range. These variable attributes permit us to group the house into several categories, which we index by $i = 1, 2, \dots, n$.

Over a period of time a number of tenants will surrender their tenancies as they move or choose to live in alternative accommodations. Furthermore, the requirements of the tenants will change with time (because new families arrive, children leave home, incomes and jobs change, and other considerations). As these changes occur, the housing authority would like to relocate each tenant to a house of his or her choice category. While the authority can often accomplish this objective by simple exchanges, it will sometimes encounter situations requiring multiple moves: moving one tenant would replace another tenant from a house in a different category, who, in turn, would replace a tenant from a house in another category, and so on, thus creating a cycle of changes. We call such a change a *cyclic change*. The decision

problem is to identify a cyclic change, if it exists, or to show that no such change exists.

To solve this problem as a network problem, we first create a *relocation graph* G whose nodes represent various categories of houses. We include arc (i, j) in the graph whenever a person living in a house of category i wishes to move to a house of category j . A directed cycle in G specifies a cycle of changes that will satisfy the requirements of one person in each of the categories contained in the cycle. Applying this method iteratively, we can satisfy the requirements of an increasing number of persons.

This application requires a method for identifying directed cycles in a network, if they exist. A well-known method, known as *topological sorting*, will identify such cycles. We discuss topological sorting in Section 3.4. In general, many tenant reassignments might be possible, so the relocation graph G might contain several cycles. In that case the authority's management would typically want to find a cycle containing as few arcs as possible, since fewer moves are easier to handle administratively. We can solve this problem using a shortest path algorithm (see Exercise 5.38).

Application 1.2 Assortment of Structural Steel Beams

In its various construction projects, a construction company needs structural steel beams of a uniform cross section but of varying lengths. For each $i = 1, \dots, n$, let $D_i > 0$ denote the demand of the steel beam of length L_i , and assume that $L_1 < L_2 < \dots < L_n$. The company could meet its needs by maintaining and drawing upon an inventory of stock containing exactly D_i units of the steel beam of length L_i . It might not be economical to carry all the demanded lengths in inventory, however, because of the high cost of setting up the inventory facility to store and handle each length. In that case, if the company needs a beam of length L_i not carried in inventory, it can cut a beam of longer length down to the desired length. The cutting operation will typically produce unusable steel as scrap. Let K_i denote the cost for setting up the inventory facility to handle beams of length L_i , and let C_i denote the cost of a beam of length L_i . The company wants to determine the lengths of beams to be carried in inventory so that it will minimize the total cost of (1) setting up the inventory facility, and (2) discarding usable steel lost as scrap.

We formulate this problem as a shortest path problem as follows. We construct a directed network G on $(n + 1)$ nodes numbered $0, 1, 2, \dots, n$; the nodes in this network correspond to various beam lengths. Node 0 corresponds to a beam of length zero and node n corresponds to the longest beam. For each node i , the network contains a directed arc to every node $j = i + 1, i + 2, \dots, n$. We interpret the arc (i, j) as representing a storage strategy in which we hold beams of length L_j in inventory and use them to satisfy the demand of all the beams of lengths $L_{i+1}, L_{i+2}, \dots, L_j$. The cost c_{ij} of the arc (i, j) is

$$c_{ij} = K_j + C_j \sum_{k=i+1}^j D_k.$$

The cost of arc (i, j) has two components: (1) the fixed cost K_j of setting up the inventory facility to handle beams of length L_j , and (2) the cost of using beams of length L_j to meet the demands of beams of lengths $L_{i+1}, \dots, L_j$. A directed path

from node 0 to node n specifies an assortment of beams to carry in inventory and the cost of the path equals the cost associated with this inventory scheme. For example, the path 0–4–6–9 corresponds to the situation in which we set up the inventory facility for handling beams of lengths L_4 , L_6 , and L_9 . Consequently, the shortest path from node 0 to node n would prescribe the least cost assortment of structural steel beams.

Application 1.3 Tournament Problem

Consider a round-robin tournament between n teams, assuming that each team plays against every other team c times. Assume that no game ends in a draw. A person claims that α_i for $1 \leq i \leq n$ denotes the number of victories accrued by the i th team at the end of the tournament. How can we determine whether the given set of non-negative integers $\alpha_1, \alpha_2, \dots, \alpha_n$ represents a possible winning record for the n teams?

Define a directed network $G = (N, A)$ with node set $N = \{1, 2, \dots, n\}$ and arc set $A = \{(i, j) \in N \times N : i < j\}$. Therefore, each node i is connected to the nodes $i + 1, i + 2, \dots, n$. Let x_{ij} for $i < j$ represent the number of times team i defeats team j . Observe that the total number of times team i beats teams $i + 1, i + 2, \dots, n$ is $\sum_{\{j: (i, j) \in A\}} x_{ij}$. Also observe that the number of times that team i beats a team $j < i$ is $c - x_{ji}$. Consequently, the total number of times that team i beats teams $1, 2, \dots, i - 1$ is $(i - 1)c - \sum_{\{j: (j, i) \in A\}} x_{ji}$. The total number of wins of team i must equal the total number of times it beats the teams $1, 2, \dots, n$. The preceding observations show that

$$\sum_{\{j: (i, j) \in A\}} x_{ij} - \sum_{\{j: (j, i) \in A\}} x_{ji} = \alpha_i - (i - 1)c \quad \text{for all } i \in N. \quad (1.3)$$

In addition, a possible winning record must also satisfy the following lower and upper bound conditions:

$$0 \leq x_{ij} \leq c \quad \text{for all } (i, j) \in A. \quad (1.4)$$

This discussion shows that the record α_i is a possible winning record if the constraints defined by (1.3) and (1.4) have a feasible solution x . Let $b(i) = \alpha_i - (i - 1)c$. Observe that the expressions $\sum_{i \in N} \alpha_i$ and $\sum_{i \in N} (i - 1)c$ are both equal to $cn(n - 1)/2$, which is the total number of games played. Consequently, $\sum_{i \in N} b(i) = 0$. The problem of finding a feasible solution of a network flow system like (1.3) and (1.4) is called a *feasible flow problem* and can be solved by solving a maximum flow problem (see Section 6.2).

Application 1.4 Leveling Mountainous Terrain

This application was inspired by a common problem facing civil engineers when they are building road networks through hilly or mountainous terrain. The problem concerns the distribution of earth from high points to low points of the terrain to produce a leveled roadbed. The engineer must determine a plan for leveling the route by

specifying the number of truckloads of earth to move between various locations along the proposed road network.

We first construct a *terrain graph*: it is an undirected graph whose nodes represent locations with a demand for earth (low points) or locations with a supply of earth (high points). An arc of this graph represents an available route for distributing the earth, and the cost of this arc represents the cost per truckload of moving earth between the two points. (A *truckload* is the basic unit for redistributing the earth.) Figure 1.2 shows a portion of the terrain graph.

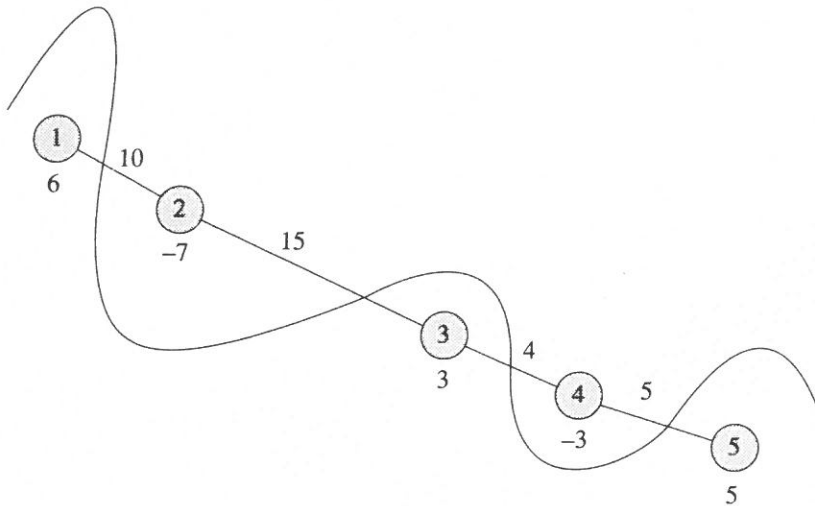


Figure 1.2 Portion of the terrain graph.

A leveling plan for a terrain graph is a flow (set of truckloads) that meets the demands at nodes (levels the low points) by the available supplies (by earth obtained from high points) at the minimum cost (for the truck movements). This model is clearly a minimum cost flow problem in the terrain graph.

Application 1.5 Rewiring of Typewriters

For several years, a company had been using special electric typewriters to prepare punched paper tapes to enter data into a digital computer. Because the typewriter is used to punch a six-hole paper tape, it can prepare $2^6 = 64$ binary hole/no-hole patterns. The typewriters have 46 characters, and each punches one of the 64 patterns. The company acquired a new digital computer that uses a different coding hole/no-hole patterns to represent characters. For example, using 1 to represent a hole and 0 to represent a no-hole, the letter A is 111100 in the code for the old computer and 011010 in the code for the new computer. The typewriter presently punches the former and must be modified to punch the latter.

Each key in the typewriter is connected to a steel code bar, so changing the code of that key requires mechanical changes in the steel bar system. The extent of the changes depends on how close the new and old characters are to each other. For the letter A, the second, third, and sixth bits are identical in the old and new codes and no changes need be made for these bits; however, the first, fourth, and fifth bits are different, so we would need to make three changes in the steel code bar connected to the A-key. Each change involves removing metal at one place and

adding metal at another place. When a key is pressed, its steel code bar activates six cross-bars (which are used by all the keys) that are connected electrically to six hole punches. If we interchange the fourth and fifth wires of the cross-bars to the hole punches (which is essentially equivalent to interchanging the fourth and fifth bits of all characters in the old code), we would reduce the number of mechanical changes needed for the A-key from three to one. However, this change of wires might increase the number of changes for some of the other 45 keys. The problem, then, is how to optimally connect the wires from the six cross-bars to the six punches so that we can minimize the number of mechanical changes on the steel code bars.

We formulate this problem as an assignment problem as follows. Define a network $G = (N_1 \cup N_2, A)$ with node sets $N_1 = \{1, 2, \dots, 6\}$ and $N_2 = \{1', 2', \dots, 6'\}$, and an arc set $A = N_1 \times N_2$; the cost of the arc $(i, j') \in A$ is the number of keys (out of 46) for which the i th bit in the old code differs from the j th bit in the new code. Thus if we assign cross-bar i to the punch j , the number of mechanical changes needed to print the i th bit of each symbol correctly is c_{ij} . Consequently, the minimum cost assignment will minimize the number of mechanical changes.

Application 1.6 Pairing Stereo Speakers

As a part of its manufacturing process, a manufacturer of stereo speakers must pair individual speakers before it can sell them as a set. The performance of the two speakers depends on their frequency response. To measure the quality of the pairs, the company generates matching coefficients for each possible pair. It calculates these coefficients by summing the absolute differences between the responses of the two speakers at 20 discrete frequencies, thus giving a matching coefficient value between 0 and 30,000. Bad matches yield a large coefficient, and a good pairing produces a low coefficient.

The manufacturer typically uses two different objectives in pairing the speakers: (1) finding as many pairs as possible whose matching coefficients do not exceed a specification limit, or (2) pairing speakers within specification limits to minimize the total sum of the matching coefficients. The first objective minimizes the number of pairs outside specification, and so the number of speakers that the firm must sell at a reduced price. This model is an application of the nonbipartite cardinality matching problem on an undirected graph: the nodes of this graph represent speakers and arcs join two nodes if the matching coefficients of the corresponding speakers are within the specification limit. The second model is an application of the nonbipartite weighted matching problem.

Application 1.7 Measuring Homogeneity of Bimetallic Objects

This application shows how a minimum spanning tree problem can be used to determine the degree to which a bimetallic object is homogeneous in composition. To use this approach, we measure the composition of the bimetallic object at a set of sample points. We then construct a network with nodes corresponding to the sample

points and with an arc connecting physically adjacent sample points. We assign a cost with each arc (i, j) equal to the product of the physical (Euclidean) distance between the sample points i and j and a homogeneity factor between 0 and 1. This homogeneity factor is 0 if the composition of the corresponding samples is exactly alike, and is 1 if the composition is very different; otherwise, it is a number between 0 and 1. Note that this measure gives greater weight to two points if they are different and are far apart. The cost of the minimum spanning tree is a measure of the homogeneity of the bimetallic object. The cost of the tree is 0 if all the sample points are exactly alike, and high cost values imply that the material is quite nonhomogeneous.

Application 1.8 Electrical Networks

The electrical network shown in Figure 1.3 has eight resistors, two current sources (at nodes 1 and 6), and one current sink (at node 7). In this network we wish to determine the equilibrium current flows through the resistors. A popular method for solving this problem is to introduce a variable x_{ij} representing the current flow on the arc (i, j) of the electrical network and write a set of equilibrium relationships for these flows; that is, the voltage–current relationship equations (using Ohm’s law) and the current balance equations (using Kirchhof’s law). The solution of these equations gives the arc currents x_{ij} . An alternative, and possibly more efficient approach is to formulate this problem as a convex cost flow problem. This formulation uses the well-known result that the equilibrium currents on resistors are those flows for which the resistors dissipate the least amount of the total power supplied by the voltage sources (i.e., the electric current follows the path of least resistance). Ohm’s law shows that a resistor of resistance r_{ij} dissipates $r_{ij}x_{ij}^2$ watts of power. Therefore, we can obtain the optimal currents by solving the following convex cost flow problem:

$$\text{Minimize } \sum_{(i,j) \in A} r_{ij}x_{ij}^2$$

subject to

$$\sum_{\{j: (i,j) \in A\}} x_{ij} - \sum_{\{j: (j,i) \in A\}} x_{ji} = b(i) \quad \text{for each node } i \in N,$$

$$x_{ij} \geq 0 \quad \text{for each arc } (i,j) \in A.$$

In this model $b(i)$ represents the supply/demand of a current source or sink.

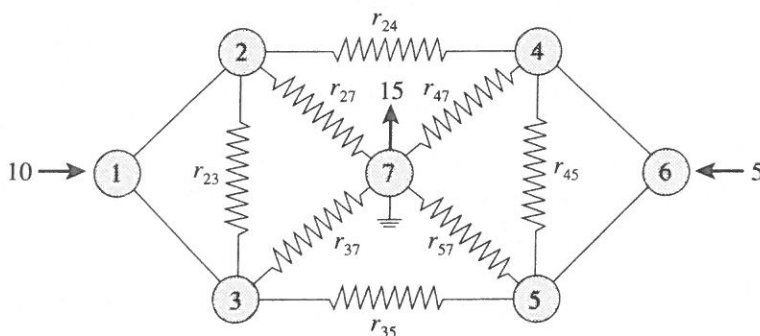


Figure 1.3 Electrical network.

The formulation of a set of equilibrium conditions as an equivalent optimization model is a powerful idea in the physical sciences, dating from the last century, which has become known as so-called *variational principles*. The term “variational” arises because the equilibrium conditions are the “optimality conditions” for the equivalent optimization model that tell us that we cannot improve the optimal solution by varying (hence the term “variational”) the optimal solution to this optimization model.

Application 1.9 Determining an Optimal Energy Policy

As part of their national planning effort, most countries need to decide on an energy policy (i.e., how to utilize the available raw materials to satisfy their energy needs). Assume, for simplicity, that a particular country has four basic raw materials: crude oil, coal, uranium, and hydropower; and it has four basic energy needs: electricity, domestic oil, petroleum, and gas. The country has the technological base and infrastructure to convert each raw material into one or more energy forms. For example, it can convert crude oil into domestic oil or petrol, coal into electricity, and so on. The available technology base specifies the efficiency and the cost of each conversion. The objective is to satisfy, at the least possible cost of energy conversion, a certain annual consumption level of various energy needs from a given annual production of raw materials.

Figure 1.4 shows the formulation of this problem as a generalized network flow problem. The network has three types of arcs: (1) *source arcs* (s, i) emanating from the source node s , (2) *sink arcs* (j, t) entering the sink node t , and (3) *conversion arcs* (i, j). The source arc (s, i) has a capacity equal to the availability $\alpha(i)$ of the raw material i and a flow multiplier of value 1. The sink arc (j, t) has capacity equal to the demand $\beta(j)$ of type j energy need and flow multiplier of value 1. Each conversion arc (i, j) represents the conversion of raw material i into the energy form j ; the multiplier of this arc is the efficiency of the conversion (i.e., units of energy j obtained from 1 unit of raw material i); and the cost of the arc (i, j) is the cost of this conversion. In this model, since $\alpha(i)$ is an upper bound on the use of raw material

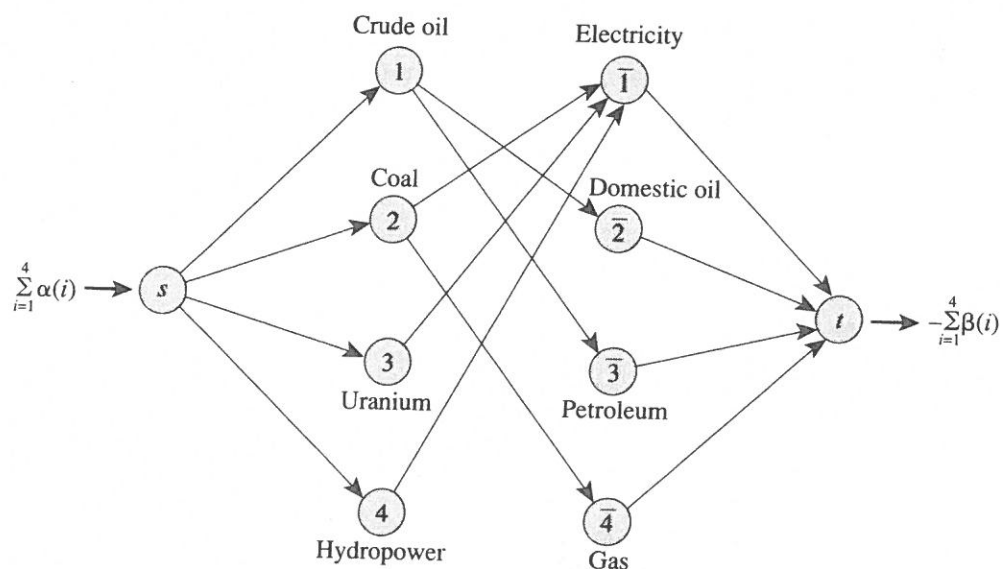


Figure 1.4 Energy problem as a generalized network flow problem.

i , $\sum_{i=1}^4 \alpha(i)$ is an upper bound on the flow out of node s . Similarly, $\sum_{i=1}^4 \beta(i)$ is a lower bound on the flow into node t . In Exercise 15.29, we show how to convert this problem into a standard form without bounds on supplies and demands.

Application 1.10 Racial Balancing of Schools

In 1968, the U.S. Supreme Court ruled that all school systems in the country should begin admitting students to schools on a nondiscriminatory basis and should employ faster techniques to promote desegregated schools across the nation. This decision made it necessary for many school systems to develop radically different procedures for assigning students to schools. Since the Supreme Court did not specify what constitutes an acceptable racial balance, the individual school boards used their own best judgments to arrive at acceptable criteria on which to base their desegregation plans. This application describes a multicommodity flow model for determining an optimal assignment of students to schools that minimizes the total distance traveled by the students, given a specification of lower and upper limits on the required racial balance in each school.

Suppose that a school district has S schools and school j has capacity u_j . For the purpose of this formulation, we divide the school district into L population centers. These locations might, for example, be census tracts, bus stops, or city blocks. The only restriction on the population centers is that they be finite in number and that a single distance measure reasonably approximates the distance any student at center i must travel if he or she is assigned to school j . Let S_{ik} denote the available number of students of the k th ethnic group at the i th population center. The objective is to assign students to schools in a way that achieves the desired ethnic composition for each school and minimizes the total distance traveled by the students. Each school j has the ethnic requirement that it must enroll at least l_{jk} and no more than u_{jk} students from the k th ethnic group.

We can model this problem as a multicommodity flow problem on an appropriately defined network. Figure 1.5 shows this network representation for a problem with three population centers and three schools. This network has one node for each population center and for each school as well as a "source" and a "sink" node for each ethnic group. The flow commodities represent the students of different ethnic groups. The students of the k th ethnic group flow from source a_k to sink e_k via population center and school nodes. We set the upper bound on arc (a_k, b_i) connecting the k th ethnic group source node and the i th population center equal to S_{ik} and the cost of the arc (b_i, c_j) connecting the i th population center and j th school equal to f_{ij} , the distance between that population center and that school. By setting the capacity of the arc (c_j, d_j) equal to u_j , we ensure that the total number of students (of all ethnic groups) allocated to the j th school does not exceed the maximum student population for this school. The students of all ethnic groups must share the capacity of each school. Finally, we incorporate constraints on the ethnic compositions of the schools by setting the lower and upper bounds on the arc (d_j, e_k) equal to l_{jk} and u_{jk} . It is fairly easy to verify that the multicommodity flow problem models the racial balancing problem, so a minimum multicommodity flow will specify an optimal assignment of students to the schools.

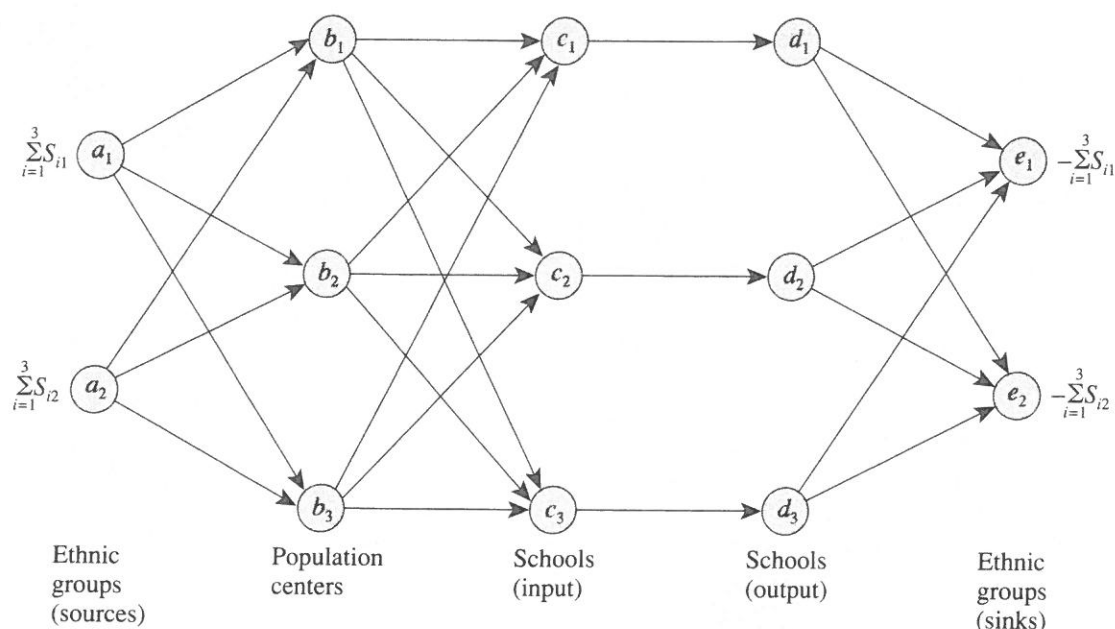


Figure 1.5 Formulating the racial balancing problem as a multicommodity flow problem.

1.4 SUMMARY

In this chapter we introduced the network flow problems that we study in this book and described a few scenarios in which these problems arise. We began by giving a linear programming formulation of the minimum cost flow problem and identifying several special cases: the shortest path problem, the maximum flow problem, the assignment problem, the transportation problem, and the circulation problem. We next described several generalizations of the minimum cost flow problem: the convex cost flow problem, the generalized network flow problem, and the multicommodity flow problem. Finally, we described two other important network models: the minimum spanning tree problem and the matching problem. Although these two problems are not network flow problems per se, we have included them in this book because they are closely related to several network flow problems and because they arise often in the context of network optimization.

Networks are pervasive and arise in numerous application settings. Physical networks, which are the most readily identifiable classes of networks, arise in many applications in many different types of systems: communications, hydraulic, mechanical, electronic, and transportation. Network flow problems also arise in surprising ways in optimization problems that on the surface might not appear to involve networks at all. We described several of these “indirect” applications of network flow problems, in such problem settings as urban housing, production planning, electrical networks, racial balancing, leveling mountainous terrain, evaluating tournaments, matching stereo speakers, wiring typewriters, assessing the homogeneity of physical materials, and energy planning. The applications we have considered offer only a brief glimpse of the wide-ranging practical importance of network flows; although our discussion of applications in this chapter is limited, it does provide at least one example of each of the network models that we have introduced in this chapter.

REFERENCE NOTES

The study of network flow models predates the development of linear programming. The first studies in this problem domain, conducted by Kantorovich [1939], Hitchcock [1941], and Koopmans [1947], considered the transportation problem, a special case of the minimum cost flow problem. These studies provided insight into the problem structure and yielded algorithmic approaches. Interest in the network flow problems grew with the advent of the simplex method by Dantzig in 1947, who also specialized this algorithm for the transportation problem (see Dantzig [1951]).

During the 1950s, researchers began to exhibit increasing interest in the minimum cost flow problem and its specializations—the shortest path problem, the maximum flow problem, and the assignment problem—mainly because of the importance of these models in real-world applications. Soon researchers developed special algorithms for solving these problems. Dantzig, Ford, and Fulkerson pioneered these efforts. Whereas Dantzig focused on the simplex-based methods, Ford and Fulkerson developed primal–dual combinatorial algorithms. The landmark books by Dantzig [1962] and Ford and Fulkerson [1962] present thorough discussions of these early contributions.

In the years following this groundbreaking work, network flow problems and their generalizations emerged as major research topics in thousands of papers and numerous text and reference books. The following books summarize developments in the field and serve as a guide to the literature:

1. *Flows in Networks* (Ford and Fulkerson [1962])
2. *Programming, Games and Transportation Networks* (Berge and Ghouila-Houri [1962])
3. *Finite Graphs and Networks* (Busacker and Saaty [1965])
4. *Network Flow, Transportation and Scheduling* (Iri [1969])
5. *Integer Programming and Network Flows* (Hu [1969])
6. *Communication, Transmission, and Transportation Networks* (Frank and Frisch [1971])
7. *Flows in Transportation Networks* (Potts and Oliver [1972])
8. *Graph Theory: An Algorithmic Approach* (Christophides [1975])
9. *Flow Algorithms* (Adel'son-Vel'ski, Dinics, and Karzanov [1975])
10. *Graph Theory with Applications* (Bondy and Murty [1976])
11. *Combinatorial Optimization: Networks and Matroids* (Lawler [1976])
12. *Optimization Algorithms for Networks and Graphs* (Minieka [1978])
13. *Graph Algorithms* (Even [1979])
14. *Algorithms for Network Programming* (Kennington and Helgason [1980])
15. *Network Flow Programming* (Jensen and Barnes [1980])
16. *Fundamentals of Network Analysis* (Phillips and Garcia-Diaz [1981])
17. *Combinatorial Optimization: Algorithms and Complexity* (Papadimitriou and Steiglitz [1982])
18. *Discrete Optimization Algorithms* (Syslo, Deo, and Kowalik [1983])
19. *Data Structures and Network Algorithms* (Tarjan [1983])

20. *Graphs and Algorithms* (Gondran and Minoux [1984])
21. *Network Flows and Monotropic Optimization* (Rockafellar [1984])
22. *Linear Programming and Network Models* (Gupta [1985])
23. *Programming in Networks and Graphs* (Derigs [1988])
24. *Linear Programming and Network Flows*, 2nd ed. (Bazaraa, Jarvis, and Sherali [1990])

As an additional source of references, the reader might consult the bibliographies on network optimization prepared by Golden and Magnanti [1977], Ahuja, Magnanti, and Orlin [1989, 1991], Bazaraa, Jarvis, and Sherali [1990], and the extensive set of references on integer programming compiled by researchers at the University of Bonn (Kastning [1976], Hausman [1978], and Von Randow [1982, 1985]).

Since the applications of network flow models are so pervasive, no single source provides a comprehensive account of network flow models and their impact on practice. Several researchers have prepared general surveys of selected application areas. Notable among these are the papers by Bennington [1974], Glover and Klingman [1976], Bodin, Golden, Assad, and Ball [1983], Aronson [1989], and Glover, Klingman, and Phillips [1990]. The book by Gondran and Minoux [1984] also describes a variety of applications of network flow problems. In this book we describe or cite over 150 selected applications of network flow problems. We provide the references for these problems in the reference notes given at the end of Chapters 4, 6, 9, 12, 13, 14, 15, 16, 17, and 19. We have adapted many of these applications from the paper of Ahuja, Magnanti, Orlin, and Reddy [1992].

The applications we present in Section 1.3 are adapted from the following references:

1. Reallocation of housing (Wright [1975])
2. Assortment of structural steel beams (Frank [1965])
3. Tournament problem (Ford and Johnson [1959])
4. Leveling mountainous terrain (Farley [1980])
5. Rewiring of typewriters (Machol [1961])
6. Pairing stereo speakers (Mason and Philpott [1988])
7. Measuring homogeneity of bimetallic objects (Shier [1982])
8. Electrical networks (Hu [1966])
9. Determining an optimal energy policy (Gondran and Minoux [1984])
10. Racial balancing of schools (Clarke and Surkis [1968])

EXERCISES

- 1.1. Formulate the following problems as circulation problems: (1) the shortest path problem; (2) the assignment problem; and (3) the transportation problem.
- 1.2. Consider a variant of the transportation problem for which (1) the sum of demands exceeds the sum of supplies, and (2) we incur a penalty p_j for every unit of unfulfilled demand at demand node j . Formulate this problem as a standard transportation problem with total supply equal to total demand.

- 1.3. In this exercise we examine a generalization of Application 1.2, concerning assortment of structural steel beams. In the discussion of that application, we assumed that if we must cut a beam of length 5 units to a length of 2 units, we obtain a single beam of length 2 units; the remaining 3 units have no value. However, in practice, from a beam of length 5 we can cut two beams of length 2; the remaining length of 1 unit will have some scrap value. Explain how we might incorporate the possibility of cutting multiple beam lengths (of the same length) from a single piece and assigning some salvage value to the scrap. Assume that the scrap has a value of β per unit length.
- 1.4. **Large-scale personnel assignment.** A recurring problem in the U.S. armed forces is efficient distribution and utilization of skilled personnel. Each month thousands of individuals in the U.S. military vacate jobs, and thousands of personnel become available for assignment. Each job has particular characteristics and skill requirements, while each person from the pool of available personnel has specific skills and preferences. Suppose that we use this information to compute the utility (or desirability) d_{ij} of each possible assignment of a person to a job. The decision problem is to assign personnel to the vacancies in a way that maximizes the total utility of all the assignments. Explain how to formulate this problem as a network flow problem.
- 1.5. **Dating problem.** A dating service receives data from p men and p women. These data determine what pairs of men and women are mutually compatible. Since the dating service's commission is proportional to the number of dates it arranges, it would like to determine the maximum number of compatible couples that can be formed. Formulate this problem as a matching problem.
- 1.6. **Pruned chessboard problem.** A chessboard consists of 64 squares arranged in eight rows and eight columns. A *domino* is a wooden or plastic piece consisting of two squares joined on a side. Show that it is possible to fully cover the chessboard using 32 dominos (i.e., each domino covers two squares of the board, no two dominos overlap, and some domino covers each square). A *pruned board* is a chessboard with some squares removed.
 - (a) Suppose that we want to know whether it is possible to fully cover a pruned board, and if not, to find the maximum number of dominos we can place on the pruned board so that each domino covers two squares and no two dominos overlap. Formulate this problem as a bipartite cardinality matching problem.
 - (b) Suppose that we prune only two diagonally opposite corners of the chessboard. Show that we cannot cover the resulting board with 31 dominos.
- 1.7. **Paragraph problem.** The well-known document processing program T_eX uses an optimization procedure to decompose a paragraph into several lines so that when lines are left- and right-adjusted, the appearance of the paragraph will be the most attractive. Suppose that a paragraph consists of n words and that each word is assigned a sequence number. Let c_{ij} denote the attractiveness of a line if it begins with the word i and ends with the word $j - 1$. The program T_eX uses formulas to compute the value of each c_{ij} . Given the c_{ij} 's, show how to formulate the problem of decomposing the paragraph into several lines of text in order to maximize the total attractiveness (of all lines) as a shortest path problem.
- 1.8. **Seat-sharing problem.** Several families are planning a shared car trip on scenic drives in the White Mountains, New Hampshire. To minimize the possibility of any quarrels, they want to assign individuals to cars so that no two members of a family are in the same car. Formulate this problem as a network flow problem.
- 1.9. **Police patrol problem** (Khan [1979]). A police department in a small city consists of three precincts denoted p_1 , p_2 , and p_3 . Each precinct is assigned a number of patrol cars equipped with two-way radios and first-aid equipment. The department operates with three shifts. Figure 1.6(a) and (b) shows the minimum and maximum number of patrol cars needed in each shift. Administrative constraints require that (1) shifts 1, 2, and 3 have, respectively, at least cars 10, 20, and 18 cars available; and (2) precincts p_1 , p_2 , and p_3 are, respectively, allocated at least 10, 14, and 13 cars. The police department wants to determine an allocation of patrol units that will meet all the require-

	Shift 1	Shift 2	Shift 3
p_1	2	4	3
p_2	3	6	5
p_3	5	7	6

(a)

	Shift 1	Shift 2	Shift 3
p_1	3	7	5
p_2	5	7	10
p_3	8	12	10

(b)

Figure 1.6 Patrol car requirements: (a) minimum required per shift; (b) maximum required per shift.

ments with the fewest possible units committed to the field. Formulate this problem as a circulation problem.

- 1.10. Forest scheduling problem.** Paper and wood products companies need to define cutting schedules that will maximize the total wood yield of their forests over some planning period. Suppose that a company with control of p forest units wants to identify the best cutting schedule over a planning horizon of k years. Forest unit i has a total acreage of a_i units, and studies that the company has undertaken predict that this unit will have w_{ij} tons of woods available for harvesting in the j th year. Based on its prediction of economic conditions, the company believes that it should harvest at least l_j tons of wood in year j . Due to the availability of equipment and personnel, the company can harvest at most u_j tons of wood in year j . Formulate the problem of determining a schedule with maximum wood yield as a network flow problem.